

Lecture 3: The Cost of an Algorithm: Bubble Sort, Big- O , and Merge Sort

Edson J. Ascencio

eja45@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

Sorting is the setting in which we learn to *price* an algorithm rather than merely check that it works. We analyse SORT, the bubble-sort procedure built in Lecture 2, and find it costs on the order of n^2 comparisons in the worst case. We then define the asymptotic notation $O(\cdot)$ that makes “on the order of” precise. Dissatisfied with n^2 , we derive MERGESORT from scratch: split the list in half, sort each half, and merge. Merging two sorted lists costs $\Theta(n)$ and the halving can be repeated only $\log_2 n$ times, so the whole procedure costs $\Theta(n \log n)$. We prove correctness by strong induction, which for MERGE requires *two* induction hypotheses and *two* base cases, one per pointer. Finally, we answer the question the lecture left open: no comparison-based sort can beat $\Theta(n \log n)$, and we show why, and say what has to be given up to go faster.

AI use declaration. Tool. Claude (Anthropic), used through Claude Code, on 2026-09-15. **What it did.** Reviewed deeply the writing style and gramatical issues of the original draft. **What I verified.** The final corrected draft and the correct implementation of suggested corrections. **Responsibility.** Any error here is mine.

1 Motivation

An algorithm that returns the right answer is not yet a usable algorithm. It also has to return it before the answer stops being useful. That second requirement is what this chapter is about.

The distinction matters in health data science because the objects we sort are rarely small. A national notifiable-disease register holds tens of millions of case records; a serological panel run

on a cohort produces one row per participant per antigen; a linkage between a birth register and a mortality register compares two lists that must first be put in a common order. In each of those tasks, sorting is not the analysis — it is the step that has to finish before the analysis can begin. An approach whose cost grows as the square of the number of records will run in a second on a class exercise of a thousand rows and will still be running tomorrow on ten million.

So we need a way to say how expensive an algorithm is that does not depend on the machine it runs on, the language it is written in, or how fast processors happen to be this year. That is the purpose of asymptotic notation, introduced in Section 4. Armed with it, we will show that the sorting procedure we built in the previous lecture is more expensive than it needs to be, and we will build a better one.

2 Setup and Notation

The problem. We are given an array A of n elements drawn from a totally ordered set — for our purposes, real numbers, though any type admitting a consistent “ $\leq$ ” comparison will do. We must rearrange A so that $A[0] \leq A[1] \leq \dots \leq A[n-1]$, keeping every element that we were given. That last clause is a real requirement and not a pedantic one: a procedure that returns a sorted array after silently dropping a record is *incorrect*, however sorted its output looks.

Indexing. Arrays are indexed from 0, so the valid positions of an array of length n are $0, 1, \dots, n-1$. We write $A[i:]$ for the subarray running from position i to the end. An index equal to the length of the array is therefore one position past the last element, which is exactly the condition we will use to detect that a list has been exhausted.

What we count. We take the *comparison* of two elements as the unit of cost, and count how many comparisons a procedure performs on an input of size n . Swaps, assignments and arithmetic on indices are counted as a constant amount of work per comparison and folded into a constant. This is the conventional accounting for sorting and it is the one used throughout [Cormen et al. \(2001\)](#).

Worst case. Cost depends on the input, not only on its size: an already-sorted array is easier to sort than a reversed one. Unless stated otherwise we report the *worst case* over all inputs of size n , written $T(n)$. The worst case is the honest number to quote when an adversary, or simply an unlucky data set, might hand you the bad input.

Definition 1 (Sorted). An array A of length n is *sorted* if $A[i] \leq A[i+1]$ for every $i \in \{0, \dots, n-2\}$. An array of length 0 or 1 is sorted vacuously.

3 Bubble Sort, and What It Costs

3.1 The procedure

Lecture 2 built the following pair of procedures. `BUBBLEUP` walks once from position j to the end of the array, swapping any adjacent pair it finds out of order; `Sort` calls it repeatedly.

Algorithm 1 `BUBBLEUP`(A, j, n) — one upward pass, from position j

```
1: if  $j \geq n - 1$  then
2:   return FALSE                                ▷ past the last adjacent pair; nothing swapped
3: end if
4:  $swapped \leftarrow \text{FALSE}$ 
5: if  $A[j] > A[j + 1]$  then
6:   exchange  $A[j]$  and  $A[j + 1]$ 
7:    $swapped \leftarrow \text{TRUE}$ 
8: end if
9:  $r \leftarrow \text{BUBBLEUP}(A, j + 1, n)$ 
10: return  $swapped \vee r$ 
```

Algorithm 2 `Sort`(A, n) — bubble sort over the first n positions

```
1: if  $n \leq 1$  then
2:   return  $A$                                     ▷ a list of one element is already sorted
3: end if
4:  $moved \leftarrow \text{BUBBLEUP}(A, 0, n)$ 
5: if  $moved = \text{FALSE}$  then
6:   return  $A$                                     ▷ a full pass with no swap: already sorted
7: end if
8: return Sort( $A, n - 1$ )
```

The image to hold on to is the one we acted out in class: a line of people of different heights, and a marker that walks from the front of the line to the back, swapping each neighbouring pair that is in the wrong order. When the marker reaches the end, the tallest person in the line has been carried all the way to the back — and is therefore in their final position.

3.2 Why it is correct

Lemma 1 (What one pass guarantees). *After `BUBBLEUP`($A, 0, n$) returns, $A[n - 1]$ holds the largest of $A[0], \dots, A[n - 1]$.*

Proof. Induct on the number of comparisons made. Before the first comparison, $A[0]$ is trivially the largest element seen so far. Suppose that when the procedure is about to compare positions j and $j + 1$, the value at position j is the largest of $A[0], \dots, A[j]$. The comparison places $\max(A[j], A[j + 1])$

at position $j + 1$, which is therefore the largest of $A[0], \dots, A[j + 1]$, re-establishing the hypothesis one position further along. When the walk stops, $j + 1 = n - 1$, so $A[n - 1]$ is the largest of the whole array. Nothing is added or removed at any step, only exchanged, so the multiset of elements is unchanged. $\square$

Proposition 2. *$\text{SORT}(A, n)$ returns a sorted permutation of its input.*

Proof. Strong induction on n . For $n \leq 1$ the array is sorted vacuously and is returned unchanged. For $n > 1$, Lemma 1 places the largest element at position $n - 1$, where it belongs in any sorted arrangement. The induction hypothesis — that SORT correctly sorts arrays of length $n - 1$ — applies to the remaining prefix $A[0..n - 2]$, which the recursive call sorts. A sorted prefix followed by an element no smaller than all of them is a sorted array. The early return is sound because a complete pass with no swap means no adjacent pair was out of order, which is the definition of sorted. $\square$

Remark 1. The early-exit test is what makes bubble sort fast on nearly-sorted input: a single pass over an already-sorted array performs $n - 1$ comparisons, finds nothing to swap, and stops. The worst case and the best case for this procedure are genuinely different, which is why the next subsection is careful to say which one it is pricing.

3.3 What it costs

Count comparisons. $\text{BUBBLEUP}(A, j, n)$ compares positions j and $j + 1$ and then calls itself on $j + 1$, so it makes $n - j - 1$ comparisons in total; starting from $j = 0$ that is $n - 1$ comparisons, and we write the cost of one full pass as $n + c$ for a constant c absorbing the bookkeeping.

In the worst case each pass moves something, so no early exit ever fires, and SORT is called on $n, n - 1, n - 2, \dots, 1$ — that is, n times. Summing the cost of the pass made at each of those calls,

$$T(n) = \sum_{k=1}^n (k + c) = \frac{n(n + 1)}{2} + cn = \frac{1}{2}n^2 + \left(\frac{1}{2} + c\right)n. \quad (1)$$

The leading term is $\frac{1}{2}n^2$. Doubling the number of records multiplies the work by roughly four. That is the behaviour we now want to name precisely, and then improve on.

4 Asymptotic Notation

Equation (1) carries more detail than we need. The constant $\frac{1}{2}$ depends on how we chose to count, and the linear term is swamped by the quadratic one as soon as n is large. What survives those arbitrary choices is the *shape* of the growth, and that is what O records.

Definition 2 (Big- O). Let T and f be functions from the positive integers to the non-negative reals. We write

$$T(n) = O(f(n))$$

if there exist a constant $c > 0$ and a threshold N such that $T(n) \leq c f(n)$ for all $n \geq N$.

Read Definition 2 as an *existence* claim, which is how it was put in class: we do not have to find the best c , or any particular c . We only have to argue that some constant multiple of f eventually sits above T and stays there. The threshold N is what licenses the phrase “for a large enough data set”: small inputs are allowed to misbehave.

Example 1. Take $T(n) = \frac{1}{2}n^2 + (\frac{1}{2} + c)n$ from (1). For $n \geq 1$ we have $n \leq n^2$, so $T(n) \leq (\frac{1}{2} + \frac{1}{2} + c)n^2 = (1 + c)n^2$. Taking the constant to be $1 + c$ and the threshold to be $N = 1$ satisfies the definition, so $T(n) = O(n^2)$.

Beyond the lecture: the other two symbols. O gives an upper bound only. Saying that bubble sort is $O(n^2)$ does not forbid it from sometimes being faster, and indeed on sorted input it is not. Two companion symbols make the other statements available, and the distinction between them is the source of a great deal of sloppy writing in the applied literature. We write $T(n) = \Omega(f(n))$ when f is a lower bound, that is, when there are $c > 0$ and N with $T(n) \geq c f(n)$ for all $n \geq N$; and $T(n) = \Theta(f(n))$ when both hold at once, so that f describes the growth exactly up to constants. The standard modern treatment of the three symbols, including the argument for using them consistently, is [Knuth \(1976\)](#).

Remark 2. It is correct but uninformative to say that bubble sort is $O(n^3)$: an upper bound that is not tight is still an upper bound. When a bound is claimed to be tight, Θ is the symbol that says so.

5 Merge Sort

5.1 The idea

Bubble sort shrinks the problem by one element per round, which is why the cost accumulates as a sum $1 + 2 + \dots + n$. Suppose instead we cut the problem in *half* at each step. Split the array into a left and a right part, sort each part, and then combine the two sorted parts into one sorted whole. If the combining step is cheap, the halving should buy us a great deal.

This is MERGESORT, attributed to John von Neumann in 1945; [Knuth \(1998\)](#) gives the history along with a detailed analysis.

Why the floor. The split point is written $\lfloor n/2 \rfloor$, the greatest integer not exceeding $n/2$, because an array index must be an integer and n need not be even. For $n = 5$ we get $\lfloor 5/2 \rfloor = 2$, so the halves

Algorithm 3 MERGESORT(A, n)

```
1: if  $n \leq 1$  then
2:   return  $A$ 
3: end if
4:  $A_1 \leftarrow A[0 : \lfloor n/2 \rfloor]$ 
5:  $A_2 \leftarrow A[\lfloor n/2 \rfloor : n]$ 
6:  $A_1 \leftarrow \text{MERGESORT}(A_1, \lfloor n/2 \rfloor)$ 
7:  $A_2 \leftarrow \text{MERGESORT}(A_2, n - \lfloor n/2 \rfloor)$ 
8: return MERGE( $A_1, A_2, 0, 0$ )
```

have lengths 2 and 3. The ceiling $\lceil 5/2 \rceil = 3$ would split them the other way. Either choice works; what does not work is writing $n/2$ and hoping. This also disposes of a natural worry: the procedure does not require n to be a power of two.

5.2 Merging, and its two base cases

MERGE takes two *already sorted* arrays and one index into each. It compares the two heads, emits the smaller, advances that one index, and recurses. This is the step we performed physically in class, with one volunteer stepping forward from each group.

Algorithm 4 MERGE(A_1, A_2, i, j) — merge two sorted arrays from positions i and j

```
1: if  $i = \text{LENGTH}(A_1)$  then
2:   return  $A_2[j : ]$                                  $\triangleright$  base case 1: left list exhausted
3: end if
4: if  $j = \text{LENGTH}(A_2)$  then
5:   return  $A_1[i : ]$                                  $\triangleright$  base case 2: right list exhausted
6: end if
7: if  $A_2[j] < A_1[i]$  then
8:    $rest \leftarrow \text{MERGE}(A_1, A_2, i, j + 1)$ 
9:   return  $A_2[j] \frown rest$ 
10: else
11:    $rest \leftarrow \text{MERGE}(A_1, A_2, i + 1, j)$ 
12:   return  $A_1[i] \frown rest$ 
13: end if
```

Here $\frown$ denotes putting an element at the front of a list, so that $x \frown (y, z) = (x, y, z)$. It is a concatenation, not an addition.

Remark 3 (Two pointers, two base cases). This was the point of the lecture that is easiest to get wrong. MERGE advances two indices independently, and either list may run out first. Both exhaustion conditions must therefore be tested, and each returns *the remainder of the other list*, which is already sorted and can be emitted wholesale. Writing only one of the two base cases produces a procedure

that is correct on many inputs and silently drops the tail of a list on the rest — exactly the class of defect that a test suite built from convenient examples will not catch.

Remark 4 (Why the index equals the length). An index equal to $\text{LENGTH}(A_1)$ is one position *past* the last element, so the test $i = \text{LENGTH}(A_1)$ fires precisely when nothing is left. The strictly weaker test $i > \text{LENGTH}(A_1)$ would skip the final element, and $i = \text{LENGTH}(A_1) - 1$ would stop one element early. This question was raised in class and it is worth restating: when the array is $(1, 2, 3)$, the valid indices are 0, 1, 2, and $i = 3$ is the first value that means “exhausted”.

5.3 Why it is correct

Lemma 3 (MERGE is correct). *If $A_1[i :]$ and $A_2[j :]$ are each sorted, $\text{MERGE}(A_1, A_2, i, j)$ returns a sorted array containing exactly the elements of $A_1[i :]$ together with those of $A_2[j :]$.*

Proof. Induct on $m = (\text{LENGTH}(A_1) - i) + (\text{LENGTH}(A_2) - j)$, the number of elements still to be emitted, which strictly decreases at every recursive call.

Base cases. If $i = \text{LENGTH}(A_1)$ the left list is empty and the procedure returns $A_2[j :]$, which is sorted by assumption and is exactly the required multiset. The case $j = \text{LENGTH}(A_2)$ is symmetric. Both are needed; see Remark 3.

Inductive step. Suppose the claim holds for every pair of indices with fewer than m elements remaining — note that this is *two* hypotheses, one for advancing i and one for advancing j , because the two pointers move independently. Let $x = \min(A_1[i], A_2[j])$. Since both lists are sorted, every remaining element of either list is at least x , so x is the minimum of the whole remaining multiset and belongs at the front of the output. The procedure emits x and recurses on the pair with that pointer advanced, which has $m - 1$ elements remaining; by hypothesis that call returns a sorted arrangement of the rest. Prefixing the overall minimum to a sorted arrangement of the remainder yields a sorted arrangement of the whole. $\square$

Theorem 4. $\text{MERGESORT}(A, n)$ returns a sorted permutation of A .

Proof. Strong induction on n . For $n \leq 1$ the array is sorted vacuously. For $n > 1$, both halves have length strictly less than n — this is where the recursion is guaranteed to make progress — so by the induction hypothesis the two recursive calls return sorted arrangements of them. Lemma 3 then applies, and its output is a sorted arrangement of the union, which is the whole of A . $\square$

Remark 5. The structure of this argument is worth separating from its content. **MERGE** is an *auxiliary* procedure, not part of the recursion of **MERGESORT**, so its correctness and its termination have to be established on their own before the main induction may lean on them. Assuming a helper works because the procedure that calls it works is circular.

5.4 What it costs

Theorem 5. *MERGESORT performs $\Theta(n \log n)$ comparisons in the worst case.*

Proof. Each call to MERGE emits one element per comparison, so merging two lists of combined length n costs at most $n - 1$ comparisons, that is $\Theta(n)$. Writing $T(n)$ for the cost of MERGESORT on n elements,

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n), \quad T(1) = \Theta(1).$$

Now count levels, which is the argument made in class. At the top level there is one problem of size n ; below it two of size $n/2$; below that four of size $n/4$. At every level the sizes sum to n , so the merging done at each level costs $\Theta(n)$ in total. The recursion stops when the pieces have size 1. If d is the number of halvings needed, then $2^d \geq n$, hence $d \geq \log_2 n$, and the number of levels is $\lceil \log_2 n \rceil$. Multiplying $\Theta(n)$ per level by $\Theta(\log n)$ levels gives $\Theta(n \log n)$. A formal treatment via the master theorem appears in [Cormen et al. \(2001\)](#). $\square$

Remark 6 (Where the logarithm comes from). The logarithm is not an artefact of the algebra. It counts *how many times a number can be halved before reaching one*, and that is what the divide-and-conquer structure does to the problem. Bubble sort removes one element per round and pays n rounds; merge sort removes half the problem per level and pays $\log_2 n$ levels. The base of the logarithm is 2 because we split in two, and changing the base changes only a constant factor, which Θ ignores.

6 Beyond the Lecture

6.1 Can we do better than $n \log n$?

The lecture closed by asking whether any sorting algorithm can run in linear time, and said the answer is no. Here is why, because the argument is short and it is one of the genuinely satisfying results in the subject.

Theorem 6 (Comparison lower bound). *Any sorting algorithm that gains information about the input only by comparing pairs of elements requires $\Omega(n \log n)$ comparisons in the worst case.*

Proof sketch. Model the algorithm as a binary decision tree. Each internal node is one comparison and has two children, one per outcome; each leaf is a possible output ordering. To be correct the algorithm must be able to produce any of the $n!$ permutations of the input, so the tree has at least $n!$ leaves. A binary tree of height h has at most 2^h leaves, so $2^h \geq n!$, giving $h \geq \log_2(n!)$. By Stirling's approximation $\log_2(n!) = \Theta(n \log n)$. The height of the tree is the number of comparisons made on the worst-case input, so that number is $\Omega(n \log n)$. $\square$

Theorem 6 is proved in full in [Cormen et al. \(2001, Ch. 8\)](#). Together with Theorem 5 it says something strong: merge sort is not merely good, it is *asymptotically optimal* among comparison sorts. There is no cleverer comparison-based algorithm waiting to be discovered.

6.2 What has to be given up to go faster

The lower bound constrains algorithms that only *compare*. Sorts that exploit the structure of the keys themselves escape it. Counting sort, given n integer keys drawn from a known range of size k , runs in $\Theta(n + k)$ by tallying occurrences; radix sort extends this to longer keys digit by digit. Both are described in [Cormen et al. \(2001, Ch. 8\)](#). They are not counterexamples to Theorem 6 — they simply do something other than comparing, and they buy their speed with an assumption about the keys and with extra memory.

This trade is directly relevant to our setting. Age in completed years, ICD chapter, epidemiological week and administrative division codes are all small-range integer keys, and can be sorted in linear time. Floating-point exposure values and free-text names cannot.

6.3 Two properties that matter in practice

Stability. A sort is *stable* if records comparing equal keep their original relative order. MERGE as written in Algorithm 4 takes from A_2 only on the strict inequality $A_2[j] < A_1[i]$, and otherwise takes from A_1 ; since A_1 holds the earlier half of the original array, ties resolve in favour of the earlier element and the procedure is stable. Reversing that comparison to $\leq$ would silently destroy stability. Stability is what allows sorting on one key and then another to produce a correct lexicographic ordering, which is the usual way a record linkage is staged.

Space. Bubble sort rearranges the array in place, using $O(1)$ extra memory. Merge sort as presented builds new arrays and needs $\Theta(n)$ auxiliary space. On data that does not comfortably fit in memory, that constant is not a detail. This is one reason production sorts are hybrids: CPython's `list.sort` and Java's sort for objects both use a merge-based algorithm adapted to exploit runs that are already ordered, precisely because real data is rarely in random order. [Sedgewick and Wayne \(2011\)](#) discusses these engineering choices at length.

7 Worked Example

Sort $A = (5, 2, 4, 6, 1, 3)$ with MERGESORT. Here $n = 6$ and $\lfloor 6/2 \rfloor = 3$.

1. Split into $(5, 2, 4)$ and $(6, 1, 3)$.
2. Sorting $(5, 2, 4)$: split into (5) and $(2, 4)$. The singleton is sorted. Sorting $(2, 4)$: split into (2) and (4) , merge to $(2, 4)$. Merge (5) with $(2, 4)$: compare 5 against 2, emit 2; compare 5

against 4, emit 4; the right list is now exhausted, so base case 2 returns the remainder of the left list, (5). Result (2, 4, 5).

3. Sorting (6, 1, 3) symmetrically gives (1, 3, 6).
4. Merge (2, 4, 5) with (1, 3, 6): emit 1, then 2, then 3, then 4, then 5; the left list is now exhausted, so base case 1 returns the remainder of the right list, (6).

The result is (1, 2, 3, 4, 5, 6). Notice that each of the two base cases fired once. A version of MERGE written with only base case 1 would have produced (2, 4) at step 2, losing the element 5 — and the final output would have been sorted, and wrong.

8 Summary

- Correctness and cost are separate obligations. Bubble sort is correct and costs $\Theta(n^2)$ in the worst case.
- $T(n) = O(f(n))$ asserts the *existence* of a constant c and a threshold N with $T(n) \leq cf(n)$ for all $n \geq N$. Ω is the matching lower bound and Θ is both.
- Merge sort splits at $\lfloor n/2 \rfloor$, sorts each half recursively and merges in linear time, for $\Theta(n \log n)$ overall. The logarithm counts halvings.
- MERGE needs *two* base cases because it advances two pointers independently. One base case yields a procedure that loses records without looking broken.
- No comparison-based sort can beat $\Theta(n \log n)$, by a decision-tree argument. Linear-time sorts exist only for keys with exploitable structure, and pay for it in assumptions and memory.
- Merge sort is stable as written and uses $\Theta(n)$ auxiliary space; bubble sort is in place.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2001). *Introduction to Algorithms*, 2nd edition. MIT Press, Cambridge, MA.
- Knuth, D. E. (1976). Big Omicron and big Omega and big Theta. *ACM SIGACT News*, 8(2):18–24.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd edition. Addison-Wesley, Reading, MA.
- Sedgewick, R. and Wayne, K. (2011). *Algorithms*, 4th edition. Addison-Wesley, Upper Saddle River, NJ.